

Big Order Bubbles [Trading IQ]

// This Pine Script™ code is subject to the terms of the Mozilla Public License 2.0 at <https://mozilla.org/MPL/2.0/>

// © TradingIQ

//@version=6

indicator("Big Order Bubbles IQ [Trading IQ]", max_labels_count = 500, max_lines_count = 500, max_boxes_count = 500, calc_bars_count = 10000, overlay = false, dynamic_requests = true)
import kaigouthro/hsvColor/15 as kai

userTF = input×string(defval = "Auto", title = "Lower Timeframe Volume To Use (Lower = Better)", options = ["Use Live Tick (No History) - Must Use Dollar Amount", "1-Second", "1-Tick", "Auto"], group = "Main Settings")

model = input.string(defval = "Turning Points", title = "Identification Type", options = ["Turning Points", "Extreme Vol"], group = "Main Settings")

maxBubbleSize = input×int(defval = 10, minval = 5, maxval = 10, title = "Max Bubble Size", group = "Main Settings")

perclInput = input×float(defval = 98, maxval = 99.99999, minval = 1, title = "Bubbles Aggressiveness", active = userTF != "Use Live Tick (No History) - Must Use Dollar Amount", group = "Main Settings")

useDollar = input×bool(defval = false, title = "Use Minimum \$ Dollar Size For Big Order", group = "Dollar Volume Settings")

dollarAmount = input×float(minval = 0, defval = 100000, title = "Minimum \$ Dollar Size", active = useDollar, group = "Dollar Volume Settings")

storage = input×int(defval = 10, minval = 1, maxval = 200, title = "Live Tick Big Order Storage", active = userTF == "Use Live Tick (No History) - Must Use Dollar Amount", group = "Live Orders")

multiBubbles = input×bool(defval = false, title = "Allow Multiple Bubbles Per Bar", group = "Aesthetics")

bullishCol = input×color(#dbdbdb, title = "General Buy Volume Color", group = "Aesthetics")

bearishCol = input×color(color×rgb(255, 116, 116), title = "General Sell Volume Color", group = "Aesthetics")

transp = input×int(defval = 0, minval = 0, maxval = 100, title = "Bubble Transparency", group = "Aesthetics")

noNeon = input×bool(defval = false, title = "No Neon", group = "Aesthetics")

```

additional  = input×bool(defval = false, title = "Add Bubbles For Custom Symbol", group = "Use
Additional Symbol")
custom      = input×symbol(defval = "", title = "Additional Symbol", active = additional, group =
"Use Additional Symbol")
bullColAd   = input×color(defval = #ffffff, title = "Additional Symbol Bullish Color", active =
additional, group = "Use Additional Symbol")
bearColAd   = input×color(defval = #000000, title = "Additional Symbol Bearish Color", active =
additional, group = "Use Additional Symbol")

```

```

showOrderTable = input×bool(defval = true, title = "Show Big Order Table", active = userTF != "Use
Live Tick (No History) - Must Use Dollar Amount", group = "Order Table Settings")
orderSizeShow = input×int(defval = 5, minval = 1, maxval = 50, title = "Number Of Orders In Table",
active = showOrderTable, group = "Order Table Settings")

```

```

type bubbleData

```

```

    float vol
    float price
    color bubbleCol
    string size

```

```

type orderData

```

```

    string price
    float  vol
    string side
    int    T
    label  bubble

```

```

type varipLab

```

```

    label varipLabInst

```

```

type liveOrderData

```

```

    float vol
    float price
    color col

```

```
int timeLab
```

```
var orderDataArr = array.new<orderData>()
```

```
var userTFde = switch userTF
```

```
    "1-Second" => "1S"
```

```
    "1-Tick"   => "1T"
```

```
    "Auto"     => ""
```

```
var finalLTF = timeframe.from_seconds(math.min(timeframe.in_seconds(userTFde),  
timeframe.in_seconds(""))))
```

```
if barstate.isfirst
```

```
    if timeframe.isticks or finalLTF == "0D" or na(finalLTF)
```

```
        finalLTF := userTFde
```

```
direction() =>
```

```
    if finalLTF == "1T"
```

```
        switch
```

```
            close == bid => -1
```

```
            close == ask => 1
```

```
            =>          math.sign(close - close[1])
```

```
    else
```

```
        math.sign(close - close[1])
```

```
main(string model, float percInput, color bullCol, color bearCol, bool isCustom) =>
```

```
[ltfvol, ltfhlc3, co, timeArr] = request.security_lower_tf(isCustom ? custom : syminfo.tickerid,
```

```

userTF != "Auto" ? finalLTF : timeframe.from_seconds(math.max(60,
timeframe.in_seconds(timeframe.period) / 5)), [volume, hlc3, direction(), time], calc_bars_count =
90000)
    wpr                = request.security(isCustom ? custom : syminfo.tickerid, "", ta.wpr(14),
lookahead = barmerge.lookahead_on)

```

```

inRange = switch

```

```

    timeframe.in_seconds("") >= 86400 => time("1S", "0000-2359")
    syminfo.type == "futures"      => time("1S", "0720-1720", "America/Chicago")
    =>                             time("1S", "0000-2359")

```

```

var ltfVolMat  = matrix.new<float>(0, 0)
var ltfVolMatC = matrix.new<float>(0, 0)
turningPointVal = bubbleData.new()
var volPoints   = array.new<float>()

```

```

if barstate.isnew
    turningPointVal := bubbleData.new()

```

```

sumBigBuy = 0., sumBigSell = 0.

```

```

if volPoints.size() > 50
    volPoints.shift()

```

```

if model == "Extreme Vol"

```

```

    var volArr = array.new_float()

```

```

    highVol = switch useDollar

```

```

        false => volArr.percentile_nearest_rank(percInput)
        =>     dollarAmount

```

```

if ltfvol.size() > 0
    for [i, vol] in ltfvol

```

```
if volArr.size() >= 10000  
    volArr.shift()
```

```
volu = vol
```

```
getTime = timeArr.get(i), gethlc3 = ltfhlc3.get(i)  
isLast = true
```

```
if useDollar
```

```
    if syminfo.volumetype != "quote"
```

```
        volu *= gethlc3
```

```
volArr.push(volu)
```

```
if volArr.size() >= 500
```

```
    if volu >= highVol and co.get(i) != 0
```

```
        getDir = co.get(i),
```

```
        switch getDir
```

```
            1 => sumBigBuy += volu
```

```
            => sumBigSell -= volu
```

```
[colx, directionStr] = switch
```

```
    getDir > 0 => [bullCol, "Long"]
```

```
    getDir < 0 => [bearCol, "Short"]
```

```
    => [color.gray, "Flat"]
```

```
volPoints.push(math.abs(volu))
```

```
min    = volPoints.min(), rangeVol = volPoints.range()
```

```
pushSize = 1 + ((math×abs(volu) - min) × (maxBubbleSize - 1)) / rangeVol
```

```
orderDataArr.unshift(  
    orderData.new(  
        str.toString(geThlc3, format.mintick),  
        volu,  
        directionStr,  
        timeArr.get(i),  
        multiBubbles ? label.new(bar_index, geThlc3, text = "●", tooltip =  
str.toString(volu, format.volume), color = #00000000, style = label.style_label_center, force_overlay  
= true, textcolor = color.new(colx, transp), size = int(pushSize + 5)) : na  
  
    ))
```

```
if orderDataArr.size() > orderSizeShow  
    orderDataArr.pop()
```

```
if volu >= nz(turningPointVal.vol)
```

```
    turningPointVal.vol      := volu  
    turningPointVal.price    := geThlc3  
    turningPointVal.bubbleCol := colx  
    turningPointVal.size     := str.toString(pushSize)
```

```
if model == "Turning Points"
```

```
    var volArr = array.new_float()
```

```
if ltfvol.size() > 0
```

```
    ltfVolMat .add_row(ltfVolMat.rows())
```

```
    ltfhlc3Sz = ltfhlc3.size() - 1
```

```
    for [i, vol] in ltfvol
```

```
volu = vol, gethlc3 = ltfhlc3.get(i)
```

```
if useDollar
```

```
    if syminfo.volumetype != "quote"
```

```
        volu *= gethlc3
```

```
if i < ltfhlc3Sz
```

```
    if i + 1 > ltfVolMat.columns()
```

```
        ltfVolMat.add_col(ltfVolMat.columns())
```

```
    if ltfVolMat.rows() > 0
```

```
        ltfVolMat.set(ltfVolMat.rows() - 1, i, volu)
```

```
    if ltfVolMat.elements_count() > 20000
```

```
        ltfVolMat.remove_row(0)
```

```
    if volArr.size() >= 250
```

```
        volArr.shift()
```

```
    getTime = timeArr.get(i)
```

```
    volArr.push(volu)
```

```
    if volArr.size() >= 20 and (wpr >= -20 or wpr <= -80)
```

```
        [preSize, comapre] = switch useDollar
```

```
            false => [volArr.percentrank(volArr.size() - 1), percInput]
```

```
            => [volu, dollarAmount]
```

```
    if preSize >= comapre
```

```
        getco = coxget(i)
```

```
if getco != 0
```

```
    switch getco
```

```
        1 => sumBigBuy += volu
```

```
        => sumBigSell -= volu
```

```
    getDir = getco,
```

```
    [colx, directionStr] = switch
```

```
        getDir > 0 => [bullCol, "Long"]
```

```
        getDir < 0 => [bearCol, "Short"]
```

```
        => [color.gray, "Flat"]
```

```
    volPoints.push(math.abs(volu))
```

```
    min = volPoints×min(), rangeVol = volPoints×range()
```

```
    pushSize = 1 + ((math×abs(volu) - min) × (maxBubbleSize - 1)) / rangeVol
```

```
    orderDataArr.unshift(
```

```
        orderData.new(
```

```
            str.toString(gethlc3, format.mintick),
```

```
            volu,
```

```
            directionStr,
```

```
            timeArr.get(i),
```

```
            multiBubbles ? label.new(bar_index, gethlc3, text = "●", tooltip =
```

```
str.toString(volu, format.volume), color = #00000000, style = label.style_label_center, force_overlay
```

```
= true, textcolor = color.new(colx, transp), size = int(pushSize + 5)) : na
```

```
        ))
```

```
    if orderDataArr.size() > orderSizeShow
```

```
        orderDataArr.pop()
```

```
    Range = (100 - percInput) / 5
```


size = switch

```
preSize < perclnput + Range => 0
preSize < perclnput + Range * 2 => 1
preSize < perclnput + Range * 3 => 2
preSize < perclnput + Range * 4 => 3
=> 4
```

if volu >= nz(turningPointVal.vol)

```
turningPointVal.vol      := volu
turningPointVal.price    := gethlc3
turningPointVal.bubbleCol := colx
turningPointVal.size     := str.toString(pushSize)
```

[sumBigBuy, sumBigSell, turningPointVal]

[sumBigBuy, sumBigSell, turningPointVal] = main(model, perclnput, bullishCol, bearishCol, false)

if additional

main(model, perclnput, bullColAd, bearColAd, true)

getTransp(size, ind, isMain = false) =>

if not isMain and not noNeon

start = math.round((size - 1) / 9 * 5)

switch

ind == start + 1 => 94

```
ind == start + 2 => 96
ind == start + 3 => 98
ind == start + 4 => 99
=> 100
```

else

```
switch ind == math.round(size) - 1 or ind == 0
  true => transp
  => 100
```

duration(ms) =>

```
Minute = 60 × 1000
Hour   = 60 × Minute
Day    = 24 × Hour
```

```
d = math×floor(ms / Day)
h = math×floor((ms % Day) / Hour)
m = math×floor((ms % Hour) / Minute)
```

```
result = ""
```

```
if d > 0
  result += str.toString(d) + "D"
```

```
else if h > 0
  result += str.toString(h) + "h" + str.toString(m) + "m"
```

```
else
  result += str.toString(m) + "m"
```

```
if result == "0m"
```

```
  result := "Now"
```

```
result
```

```
var useLive = userTF == "Use Live Tick (No History) - Must Use Dollar Amount"
```

```
liveOrders() =>
```

```
if barstate.islast and useLive
```

```
    varip sumBigBuyLive = 0.  
    varip sumBigSellLive = 0.  
    varipLabel = varipLab.new()  
    varip liveOrderDataN = array.new<liveOrderData>()  
    varip tickVol = float(na)  
    varip tickVolN = float(na)  
    varip Close = float(na)  
    varip CloseN = float(na)  
    varip Time = int(na)  
    varip TimeN = int(na)
```

```
if barstate.isnew  
    sumBigBuyLive := 0.  
    sumBigSellLive := 0.  
    tickVol := float(na)  
    tickVolN := 0.  
    liveOrderDataN.clear()
```

```
Time := TimeN  
TimeN := timenow
```

```
Close := CloseN  
CloseN := close
```

```
tickVol := tickVolN  
tickVolN := volume
```

```
varip isBigOrder = false
```

```
isBigOrder := false
```

```
if Time != TimeN or CloseN >= Close or CloseN < Close
```

```
    difference = tickVolN - tickVol
```

```
    if syminfo.volumetype != "quote"
```

```
        difference *= CloseN
```

```
    if difference >= dollarAmount
```

```
        isBigOrder := true
```

```
        col = color(na)
```

```
        switch
```

```
            CloseN > Close => col := bullishCol, sumBigBuyLive += difference
```

```
            CloseN < Close => col := bearishCol, sumBigSellLive -= difference
```

```
            CloseN == Close => col := color.gray
```

```
        liveOrderDataN.push(liveOrderData.new(difference, CloseN, col, time))
```

```
        if liveOrderDataN.size() > storage
```

```
            liveOrderDataN.shift()
```

```
size = liveOrderDataN.size()
```

```
if size > 0
```

```
    for data in liveOrderDataN
```

```
        varipLabel.varipLabInst := label.new(data.timeLab, data.price, text = "●", tooltip =  
str.toString(data.vol, format.volume), color = #00000000, textcolor = dataxcol, style =  
label.style_label_center, force_overlay = true, xloc = xloc.bar_time)
```

```
[isBigOrder, sumBigBuyLive, sumBigSellLive]
```

```
[isBigOrder, sumBigBuyLive, sumBigSellLive] = liveOrders()
```

```
if useLive
```

```
    sumBigBuy    := sumBigBuyLive
```

```
    sumBigSell   := sumBigSellLive
```

```
cond = switch useLive
```

```
    false => not na(turningPointVal.price)
```

```
    =>      isBigOrder
```

```
bubbleData index = na, w = float(na)
```

```
if not useLive and cond
```

```
    index := turningPointVal.copy()
```

```
    w     := str.tonumber(turningPointVal.size)
```

```
alertcondition(cond, title = "Big Order", message = "Big Order")
```

```
if barstate.islast
```

```
    if not useLive and showOrderTable
```

```
        var orderTable = table.new(position.bottom_right, 99, 99, bgcolor = #20222C, border_color =  
#363843, frame_color = #363843, force_overlay = true, border_width = 1, frame_width = 1)
```

```
        orderTable.cell(0, 0, "Side", text_color = color×white)
```

```
        orderTable.cell(1, 0, "Size", text_color = color×white)
```

```
        orderTable.cell(2, 0, "Price", text_color = color×white)
```

```
        orderTable.cell(3, 0, "Time Since", text_color = color×white)
```

```
        max = 0., min = float(na)
```

```
        if orderDataArr.size() > 0
```

```

for [i, data] in orderDataArr

    timeSince = duration(timenow - data×T)
    max      := math.max(data.vol, max)

    min := switch na(min)

        true => data.vol
        =>    math.min(data.vol, min)

    orderTable.cell(0, i + 1, data.side, text_size = 12, text_color = color(na))
    orderTable.cell(1, i + 1, str.toString(data.vol, format.volume), text_size = 12, text_color =
color(na))
    orderTable.cell(2, i + 1, data.price, text_size = 12, text_color = color(na))
    orderTable.cell(3, i + 1, timeSince, text_size = 12, text_color = color(na))

for [i, data] in orderDataArr

    col = switch data.side

        "Long" => bullishCol
        =>    bearishCol

    grad = color.from_gradient(data.vol, min, max, color.new(col, 60), col)

    for x = 0 to 3

        orderTable.cell_set_text_color(x, i + 1, grad)

var titleLab = label(na)

titleLab.delete()
titleLab := label.new(bar_index + 1, 0, text = "Big Orders On Bar", color = #00000000, textcolor =
chart.fg_color, style = label.style_label_left)

plot(sumBigBuy, style = plot.style_histogram, linewidth = 2, color = bullishCol)

```

```
plot(sumBigBuy, style = plot.style_histogram, linewidth = 5, color = color$new(bullishCol, 94),
display = display$pane)
plot(sumBigBuy, style = plot.style_histogram, linewidth = 7, color = color$new(bullishCol, 96),
display = display$pane)
plot(sumBigBuy, style = plot.style_histogram, linewidth = 9, color = color$new(bullishCol, 98),
display = display$pane)
plot(sumBigBuy, style = plot.style_histogram, linewidth = 11, color = color$new(bullishCol, 99),
display = display$pane)
```

```
plot(sumBigSell, style = plot.style_histogram, linewidth = 2, color = bearishCol)
plot(sumBigSell, style = plot.style_histogram, linewidth = 5, color = color$new(bearishCol, 94),
display = display$pane)
plot(sumBigSell, style = plot.style_histogram, linewidth = 7, color = color$new(bearishCol, 96),
display = display$pane)
plot(sumBigSell, style = plot.style_histogram, linewidth = 9, color = color$new(bearishCol, 98),
display = display$pane)
plot(sumBigSell, style = plot.style_histogram, linewidth = 11, color = color$new(bearishCol, 99),
display = display$pane)
```

plotCond = cond and not useLive and not multiBubbles

```
plot(plotCond ? index.price : na, color = plotCond ? color$new(index.bubbleCol, getTransp(w, 0,
true)) : na, style = plot.style_circles, linewidth = 1, force_overlay = true, display = display$pane)
plot(plotCond ? index.price : na, color = plotCond ? color$new(index.bubbleCol, getTransp(w, 1,
true)) : na, style = plot.style_circles, linewidth = 2, force_overlay = true, display = display$pane)
plot(plotCond ? index.price : na, color = plotCond ? color$new(index.bubbleCol, getTransp(w, 2,
true)) : na, style = plot.style_circles, linewidth = 3, force_overlay = true, display = display$pane)
plot(plotCond ? index.price : na, color = plotCond ? color$new(index.bubbleCol, getTransp(w, 3,
true)) : na, style = plot.style_circles, linewidth = 4, force_overlay = true, display = display$pane)
plot(plotCond ? index.price : na, color = plotCond ? color$new(index.bubbleCol, getTransp(w, 4,
true)) : na, style = plot.style_circles, linewidth = 5, force_overlay = true, display = display$pane)
plot(plotCond ? index.price : na, color = plotCond ? color$new(index.bubbleCol, getTransp(w, 5,
true)) : na, style = plot.style_circles, linewidth = 6, force_overlay = true, display = display$pane)
plot(plotCond ? index.price : na, color = plotCond ? color$new(index.bubbleCol, getTransp(w, 6,
true)) : na, style = plot.style_circles, linewidth = 7, force_overlay = true, display = display$pane)
plot(plotCond ? index.price : na, color = plotCond ? color$new(index.bubbleCol, getTransp(w, 7,
```

```
true)) : na, style = plot.style_circles, linewidth = 8, force_overlay = true, display = display×pane)
plot(plotCond ? index.price : na, color = plotCond ? color.new(index.bubbleCol, getTransp(w, 8,
true)) : na, style = plot.style_circles, linewidth = 9, force_overlay = true, display = display×pane)
plot(plotCond ? index.price : na, color = plotCond ? color.new(index.bubbleCol, getTransp(w, 9,
true)) : na, style = plot.style_circles, linewidth = 10, force_overlay = true, display = display×pane)
plot(plotCond ? index.price : na, color = plotCond ? color.new(index.bubbleCol, getTransp(w, 0)) :
na, style = plot.style_circles, linewidth = 2, force_overlay = true, display = display×pane)
plot(plotCond ? index.price : na, color = plotCond ? color.new(index.bubbleCol, getTransp(w, 1)) :
na, style = plot.style_circles, linewidth = 4, force_overlay = true), display = display×pane)
plot(plotCond ? index.price : na, color = plotCond ? color.new(index.bubbleCol, getTransp(w, 2)) :
na, style = plot.style_circles, linewidth = 6, force_overlay = true, display = display×pane)
plot(plotCond ? index.price : na, color = plotCond ? color.new(index.bubbleCol, getTransp(w, 3)) :
na, style = plot.style_circles, linewidth = 8, force_overlay = true, display = display×pane)
plot(plotCond ? index.price : na, color = plotCond ? color.new(index.bubbleCol, getTransp(w, 4)) :
na, style = plot.style_circles, linewidth = 10, force_overlay = true, display = display×pane)
plot(plotCond ? index.price : na, color = plotCond ? color.new(index.bubbleCol, getTransp(w, 5)) :
na, style = plot.style_circles, linewidth = 12, force_overlay = true, display = display×pane)
plot(plotCond ? index.price : na, color = plotCond ? color.new(index.bubbleCol, getTransp(w, 6)) :
na, style = plot.style_circles, linewidth = 14, force_overlay = true, display = display×pane)
plot(plotCond ? index.price : na, color = plotCond ? color.new(index.bubbleCol, getTransp(w, 7)) :
na, style = plot.style_circles, linewidth = 16, force_overlay = true, display = display×pane)
plot(plotCond ? index.price : na, color = plotCond ? color.new(index.bubbleCol, getTransp(w, 8)) :
na, style = plot.style_circles, linewidth = 18, force_overlay = true, display = display.pane)
```